

Week 3- Algorithm Efficiency and Asymptotic notation

Instructors: Dr. Fatima Adamu-Fika
and Ms. Onyinye Okpoko

Objectives

- Define algorithm efficiency and explain its importance in computing.
- Understand and use Big-O, Big-Omega, and Big-Theta notations to analyse algorithm growth rates.
- Apply asymptotic notation to analyse basic algorithm performance.
- Compare time complexities to understand scalability.

Introduction

The topic focuses on understanding how algorithms perform as they scale with larger input sizes. This is essential in computer science because, in most cases, we're not just interested in whether an algorithm works, but how efficiently it does so, especially with bigger data sets.

What is efficiency?

Algorithm efficiency is a measure of how well an algorithm utilises resources, like time and memory, as it processes inputs

This concept is vital because efficient algorithms handle larger data sizes and intensive computations more effectively, which is crucial in fields such as machine learning, data science, and software development.”

Example:

Consider a sorting algorithm. An efficient sorting algorithm can handle millions of records in seconds, whereas an inefficient one might take much longer, which is impractical in real-world applications like e-commerce or financial services.”

Why efficiency?

- Efficiency is key in today's data-driven world. Let's think about some practical examples:
- **Data Processing:** Companies handle vast volumes of data daily. Efficient algorithms allow them to process this data faster, saving time and resources. For instance, processing social media data for insights requires algorithms that can handle scale.
- **Web Search:** Search engines, like Google, index billions of web pages. Efficient search algorithms ensure that users can retrieve relevant results within seconds.

- **Cybersecurity:** In security, efficient encryption and decryption algorithms are crucial, not only for protecting data but also for ensuring system speed and responsiveness.”
- **Connect to User Experience:** If an application’s algorithms aren’t efficient, the application can run slowly, frustrate users, and ultimately affect its adoption and success. This is particularly important as data sizes increase, as we’ll soon see that some algorithms that work fine on small inputs may become unusable with larger data.

Overview of asymptotic notations

Purpose of asymptotic notations

To measure and compare algorithms, we need a universal way to describe their efficiency, especially as input size grows. This is where asymptotic notation, like Big-O, Big-Omega, and Big-Theta, comes in. These notations let us describe an algorithm's behaviour in terms of growth rates, making it easier to compare different approaches to the same problem.

For example, if you're comparing two algorithms that solve the same problem, Big-O notation can give you insight into how each algorithm will perform as the input grows.

Asymptotic notations

- **Big-O:** Represents an upper bound on the growth rate, giving us a sense of the maximum time or space an algorithm might require.
- **Big-Omega:** Describes a lower bound, indicating the minimum time or space needed in the best case.
- **Big-Theta:** Shows a 'tight' bound, meaning the growth rate is both upper and lower bounded by the same function.

Using these notations helps computer scientists and engineers worldwide speak a common language when discussing algorithm performance.

- **Big-O (O):**

Use When: Setting an upper limit on an algorithm's runtime.

What It Tells Us: Big-O describes the maximum rate at which runtime grows as input size increases, ensuring the algorithm won't take longer than this specified rate.

Example: Stating an algorithm is $O(n)$ implies its runtime grows at most linearly with input size, though it might run faster for some inputs.

- **Big-Theta (Θ):**

Use When: Describing the exact growth rate of an algorithm's runtime.

What It Tells Us: Big-Theta gives a precise rate of growth, representing both upper and lower bounds. This notation is useful when the algorithm consistently follows this rate across all cases.

Example: An algorithm that is $\Theta(n^2)$ will always scale quadratically with input size, neither faster nor slower.

- **Big-Omega (Ω):**

Use When: Setting a lower limit on an algorithm's runtime.

What It Tells Us: Big-Omega describes the minimum growth rate. This notation helps establish that the runtime won't perform better than this rate.

Example: Stating an algorithm is $\Omega(n)$ means it won't be faster than linear time for any input size, though it could be slower.

Choosing the Correct Notation

Choosing the appropriate notation clarifies an algorithm's efficiency and accurately conveys its runtime boundaries. Using the right notation can help in selecting algorithms that match specific performance needs without overestimating or underestimating their efficiency.

Which is Better?

- $O(n)$ vs $\Theta(n)$

Better Choice: $O(n)$ because it allows for the potential of faster performance on some inputs.

With $O(n)$, the algorithm won't exceed linear time but might occasionally run faster. $\Theta(n)$, however, guarantees a fixed linear runtime, offering no room for quicker execution.

- $\Theta(n)$ vs $\Omega(n)$

Better Choice: $\Theta(n)$ as it provides consistent performance, ensuring the algorithm will take linear time across all inputs. $\Omega(n)$, in contrast, only gives a minimum bound, meaning the algorithm might sometimes require more time.

- $O(n)$ vs $\Omega(n)$

Better Choice: $O(n)$ because it provides an upper limit, ensuring the algorithm won't exceed linear time. $\Omega(n)$ only specifies a lower bound, allowing for potentially slower runtimes in some cases.

Key terminologies

- **Efficiency:** Efficiency measures how well an algorithm uses resources, especially for large data.
- **Asymptotic Notation:** Big-O, Big-Omega, and Big-Theta are tools to describe algorithm growth.
- **Performance Analysis:** Applying notation helps analyse algorithms without needing extensive testing.
- **Scalability:** Comparing time complexities shows which algorithms handle large inputs effectively.

Measuring algorithm efficiency

We can measure an algorithms efficiency either empirically or theoretically.

Empirical measurement would mean to translate the algorithm into a program and use some standard unit of time as a measurement. However, this is not the best way for many reasons. To do so necessitates the implementation of the algorithm, which may be difficult to do. The running time will be affected by factors such as the speed of the computer, the programming language, the translation of the language into machine code, and so on. The results of the analysis may not be indicative of the running time on other inputs not included in the experiment. To compare two or more algorithms, the same hardware and software environments must be used.

A better approach would be the theoretical analysis. This involves using the pseudocode of the algorithm to measure the running time as a function of the input size. This allows us to consider all possible inputs. Also, it allows us to evaluate the speed of the algorithm independent of the hardware and software environment. And the most obvious benefit, we can analyse algorithms without implementing them or running experiments on them.

We can carry this out using the following steps:

- Determine its time efficiency (estimate its runtime)
- Determine its time complexity (order of growth)

Definition of Input Size

How input size (n) impacts an algorithm's efficiency and complexity:
As algorithms generally run longer on larger inputs. For example, it will take longer to sort a larger student list, compute the factorial of a larger number, multiply larger matrices, and so on. So we evaluate an algorithm's efficiency as a function of some parameter n indicating the algorithm's input size (number of elements in the input). However, some algorithms require more than one input to indicate the size of their input. For instance, if the input to an algorithm is a graph, the input size can be described by the number of vertices and edges in the graph.

Examples of Input Metrics:

- Length of an array (for searching or sorting).
- Number of nodes in a graph (for graph algorithms).

Importance of Input Size:

- Predicting runtime and performance based on input size.

Understanding input size helps us anticipate how long the algorithm will take to run and how much memory it will need, especially when dealing with large datasets. Therefore, knowing input size helps in forecasting an algorithm's runtime and scalability.

Example:

Consider a sorting algorithm that performs well on a dataset of 10 elements. If the dataset expands to 10,000 elements, the input size drastically increases, and a previously efficient algorithm may no longer be practical.

Comparing two sorting algorithms

Bubble Sort vs. Merge Sort:

To illustrate the importance of efficiency, let's look at two sorting algorithms: Bubble Sort and Merge Sort.

- **Bubble Sort:** Bubble Sort has a time complexity of $O(n^2)$. As input size n grows, Bubble Sort quickly becomes slow due to its nested loop structure.
- **Merge Sort:** Merge Sort, on the other hand, operates with a time complexity of $O(n \log n)$, allowing it to handle larger datasets much more efficiently.

Conclusion:

As input size increases, the difference in performance becomes significant. Merge Sort will continue to scale well, whereas Bubble Sort will struggle with larger inputs, demonstrating why efficient algorithms are essential.

Theoretical algorithm analysis:

Method 1

Identifying Primitive Operations

- Primitive operations are basic, low-level steps, such as additions, comparisons, assignments. They are fundamental actions, such as assignments, additions, and comparisons, that form the building blocks of algorithms.

Purpose:

Counting these operations allows us to systematically estimate an algorithm's runtime based on input size, laying the foundation for calculating time complexity.

Examples of primitive operations:

- Assigning a value to a variable e.g., $j = 2$
- Evaluating an expression e.g., $2k^2 + 1$
- Indexing into an array e.g., $A[i]$
- Comparing two values e.g., $j \leq n - 1$
- Calling a routine or a subroutine e.g., *BubbleSort*[*A*]
- Returning from a routine or subroutine e.g., *return A*

Example Exercise: Counting Operations in a Simple Loop

Algorithm SumArray(arr)

Input: arr, an array of integers with length n

Output: total, the sum of elements in arr

1. total = 0
2. for i = 0 to n - 1 do
3. total = total + arr[i]
4. return total

Step-by-step breakdown of operations

1. **Line 1:** total = 0

- **Operations:** 1 assignment.
- **Executions:** 1 (executes only once).
- **Total Units:** 1.

2. **Line 2:** for i = 0 to n - 1 do

- **Components:**
- **Initialisation** (i = 0): 1 assignment operation, which executes once.
- **Comparison** (i <= n - 1): 1 comparison operation, which executes n + 1 times.
- **Increment** (i = i + 1): 1 addition operation and 1 assignment operation (2 primitive operations), which executes n times.
- **Total Units:**
- Initialisation: 1
- Comparison: n + 1
- Increment: 2n
- **Combined Total** for Line 2: $1 + (n + 1) + 2n = 3n + 2$

3. **Line 3:** $\text{total} = \text{total} + \text{arr}[i]$

- **Operations:** 1 addition operation, 1 assignment operation and 1 array indexing operation (3 primitive operations).
- **Executions:** n times, once for each element in the array.
- **Total Units:** $3n$.

4. **Line 4:** return total

- **Operations:** 1 return operation.
- **Executions:** 1 (executes only once).
- **Total Units:** 1.

Summary of Total Primitive Operations

Adding up all contributions:

- Line 1: 1 unit
- Line 2: $3n + 2$ units
- Line 3: $3n$ units
- Line 4: 1 unit

Total Units for Algorithm: $6n + 4$.

This step-by-step counting shows how each line contributes to the overall runtime. Summing these gives us an understanding of how runtime scales with input size.

Theoretical algorithm analysis:

Method 2

Identifying Basic Operations in an Algorithm

The **basic operation** in an algorithm is the key operation that determines its overall running time. It is typically:

- The most frequently executed step.
 - The operation that dominates the runtime as input size increases.
 - Algorithm-specific and chosen to simplify the analysis.
- **Key Focus Areas:**
 - **Loops:** Determine the number of iterations and operations within each loop.
 - **Function Calls:** Understand the impact of each call on runtime.
 - **Recursive Calls:** Track recursion depth and the operations involved.

Examples:

1. Loops

- Loops are often the main factor affecting runtime, as each iteration adds to the total operations.
- Identifying the loop's primary operation helps us understand the overall complexity.

```
total = 0
```

```
for i = 0 to n - 1 do
```

```
total = total + arr[i]
```

Key Operation: Each iteration in this loop involves an addition and assignment, and the loop runs n times.

2. Functions

In function-based algorithms, calls to other functions can significantly add to runtime. Each call might include operations that impact complexity.”

Example:

```
Algorithm ComputeSum(arr)
    return SumArray(arr)
```

Key Operation: Calling SumArray includes all operations from that function.

3. Recursive Calls

For recursive algorithms, each recursive call performs work, often resulting in exponential growth if not properly limited.”

Example:

Algorithm RecursiveSum(n)

 if $n == 0$ then

 return 0

 else

 return $n + \text{RecursiveSum}(n - 1)$

Key Operation: Each recursive call executes until reaching the base case.

We will now analyse Algorithm SumArray(arr) to get its basic operation:

The **basic operation** of this algorithm is the operation that is executed most frequently and dominates the runtime. In this case, it is:

$\text{total} = \text{total} + \text{arr}[i]$

- This operation is performed once per iteration of the loop.
- Since the loop runs n times, the total number of executions of the basic operation is proportional to n .

Note: By identifying the core operations within the algorithm, we can better understand how runtime will scale with larger inputs. Recognising the operation aids in accurately estimating runtime for the entire algorithm.

We get the summation from the basic operation.

Summation Notation:

Used to represent the total operations within loops and recursive structures.

Purpose:

Summations help calculate the cumulative work done in an algorithm, especially for nested loops and recursive calls.

Single Loop Example:

Pseudocode:

total = 0

for i = 1 to n do

 total = total + i

Summation Representation:

- The loop performs n additions.
- Summation: $\sum_{i=1}^n 1 = n$.

Interpretation: Runtime grows linearly with n , giving an overall complexity of $O(n)$.

Nested Loop Example:

Pseudocode:

```
for i = 1 to n do  
    for j = 1 to n do  
        process(i, j)
```

Summation Representation:

- The inner loop executes n times for each iteration of the outer loop.
- Total operations: $\sum_{i=1}^n \sum_{j=1}^n 1 = n^2$.

Interpretation: Runtime grows quadratically with n , giving an overall complexity of $O(n^2)$.

Converting Summations to Big-O Notation

Simplifying Summations:

- After representing operations with summations, we often simplify them to Big-O notation.
- For example, $\sum_{i=1}^n 1 = n$ simplifies to $O(n)$, and $\sum_{i=1}^n \sum_{j=1}^n 1 = n^2$ simplifies to $O(n^2)$.

Order of growth

Definition of Order of Growth: The order of growth refers to the dominant term in an algorithm's complexity, representing its rate of increase as input size grows.

Why Order of Growth Matters: Understanding the growth rate helps predict how an algorithm will scale with larger inputs.

Simplifying Summations to Big-O:

Identify and retain the dominant term to simplify expressions.

Example:

- $n^2 + 3n + 5$ simplifies to $O(n^2)$ since n^2 is the largest term as n grows.

Best, Worst, and Average Cases:

- **Best Case:** Minimum amount of work an algorithm performs under ideal conditions.
- **Worst Case:** Maximum work required when the algorithm operates the least efficiently.
- **Average Case:** Expected work across typical inputs, providing a balanced view of performance.

Importance of Analysing Each Scenario:

- Understanding different scenarios allows us to predict how an algorithm will behave under varying conditions.

Example: Linear Search:

- Best Case: Target is the first element.
- Worst Case: Target is the last element or not present.
- Average Case: Target is somewhere in the middle of the list.

Common Complexity Classes

- **Constant Time – $O(1)$:** The runtime does not change with input size.
Example: Accessing an array element.
- **Logarithmic Time – $O(\log n)$:** The runtime grows logarithmically with input size.
Example: Binary search.
- **Linear Time – $O(n)$:** The runtime grows proportionally to input size.
Example: Scanning an unsorted list.
- **Log-Linear Time – $O(n \log n)$:** Grows faster than linear but slower than quadratic.
Example: Efficient sorting (e.g., Merge Sort).
- **Quadratic Time – $O(n^2)$:** Runtime grows with the square of input size.
Example: Bubble Sort with nested loops.
- **Exponential Time – $O(2^n)$:** Runtime doubles with each addition to the input size.
Example: Recursive Fibonacci.
- **Factorial Time – $O(n!)$:** Runtime grows as the factorial of the input size.
Example: Travelling Salesman Problem (brute-force solution).

Crossover Points

Definition: The point at which one algorithm outperforms another due to differences in complexity and constant factors.

Importance: Helps decide which algorithm to use based on input size and performance requirements.

Example Comparison:

Algorithm A: $O(100n)$ – High constant factor linear time.

Algorithm B: $O(n^2)$ – Quadratic time complexity.

Insight: Although $O(n^2)$ may be faster for small inputs, $O(100n)$ eventually becomes more efficient as input size grows.

Scalability:

As input size increases, algorithms with lower complexity classes typically perform better.

Choosing the right algorithm is context-dependent and should consider both input size and constant factors.

Fundamental Statement Types in Complexity

- **Sequencing:**

When we talk about sequencing, it simply means the program does one operation after another. Each operation is quick, and overall, it has constant time complexity, $O(1)$.

- **Branching:**

Branching is just deciding between two paths based on some condition. It doesn't take much time, so it is also $O(1)$. Even though it looks like the program might "branch out," it only checks the condition once.

- **Looping:**

When you see a loop, the time complexity usually depends on how many times the loop runs. If it runs n times, it will be $O(n)$. Nested loops (loops inside loops) make the complexity grow faster. So, if we have a loop inside another loop, the total complexity would be $O(n^2)$.

- **Function Calls:**

Every time a function is called, the complexity of that function is added to the overall complexity. If the function takes $O(n)$ time, then calling it will increase the overall time of the program by $O(n)$ as well.

Practical Applications of Algorithm Efficiency

Sorting and Searching:

Importance: Efficient algorithms like QuickSort and Binary Search are crucial for handling large datasets in databases, search engines, and e-commerce.

Graph Algorithms:

Application: Shortest path algorithms (e.g., Dijkstra's) are used in GPS navigation, social networks, and transportation networks.

Cryptographic Algorithms:

Application: Cryptographic algorithms rely on complexity to secure data, using time-consuming operations that deter brute-force attacks.

Machine Learning and Data Processing:

Importance: Machine learning algorithms must be efficient to handle large datasets in training and prediction tasks, such as linear regression for data analysis.

Balancing Efficiency with Complexity

- Trade-Offs: Faster algorithms may use more memory, while memory-efficient ones may be slower. Understanding these trade-offs helps in selecting the right algorithm.

Scalability in Real-World Applications:

As input data grows, algorithms with lower complexity (e.g., $O(\log n)$ or $O(n)$) scale better than higher-complexity algorithms.

Example: Choosing $O(n \log n)$ sorting algorithms like MergeSort for large datasets over $O(n^2)$ algorithms like Bubble Sort.